

LeetCode 85 & 221

1

Maximal Rectangle & Maximal Square

1. Notes & Algorithmic Analysis

DE Shaw

Wayfair

Flipkart

Intuit

Myntra

SAP

Brute Force Approach

Both problems can be solved brute-forcefully using the **2D Prefix Sum and Offset Method**.

For Maximal Rectangle (LC 85):

- Time Complexity: $O(n^2m^2)$
- Space Complexity: $O(n \times m)$

For Maximal Square (LC 221):

- Time Complexity: $O(n^2m)$
- Space Complexity: $O(n \times m)$

Optimal Approach (LC 85)

Convert every row into a **Histogram** (as done in LC 1504), and since we need the maximum area, we calculate the maximum area of that histogram (exactly the same logic as LC 84).

Key Insight: LC 85 = LC 84 + LC 1504

- Time Complexity: $O(n^2)$ (Note: practically $O(n \times m)$)
- Space Complexity: $O(n \times m)$

2. Code Implementations

Code 1: Maximal Rectangle (Brute Force)

Implemented using the 2D Prefix Sum and Offset Method.

```
class Solution {
public:
    int maximalRectangle(vector<vector<char>>& matrix) {
        int ans=0;
        int n=matrix.size();
        int m=matrix[0].size();

        vector<vector<int>> pref(n,vector<int>(m));
        for(int i=0;i<n;i++){
            for(int j=0;j<m;j++){
                pref[i][j]=matrix[i][j]-'0';
                if(i>0){
                    pref[i][j]+=pref[i-1][j];
                }
                if(j>0){
                    pref[i][j]+=pref[i][j-1];
                }
                if(i>0 && j>0){
                    pref[i][j]-=pref[i-1][j-1];
                }
            }
        }
        for(int h=1;h<=n;h++){
            for(int w=1;w<=m;w++){
                for(int roff=0;roff+h<=n;roff++){
                    for(int coff=0;coff+w<=m;coff++){
                        int r1=roff;
                        int c1=coff;
                        int r2=roff+h-1;
                        int c2=coff+w-1;
                        int sum=pref[r2][c2];
                        if(r1>0){
                            sum-=pref[r1-1][c2];
                        }
                        if(c1>0){
                            sum-=pref[r2][c1-1];
                        }
                        if(r1>0 && c1>0){
                            sum+=pref[r1-1][c1-1];
                        }
                        if(sum==w*h){
                            ans=max(ans,sum);
                        }
                    }
                }
            }
        }
        return ans;
    }
};
```

```
    }
};
```

Code 2: Maximal Square (Brute Force)

Implemented using the 2D Prefix Sum and Offset Method tailored for squares.

```
class Solution {
public:
    int maximalSquare(vector<vector<char>>& matrix) {
        int ans=0;
        int n=matrix.size();
        int m=matrix[0].size();

        vector<vector<int>> pref(n,vector<int>(m));
        for(int i=0;i<n;i++){
            for(int j=0;j<m;j++){
                pref[i][j]=matrix[i][j]-'0';
                if(i>0){
                    pref[i][j]+=pref[i-1][j];
                }
                if(j>0){
                    pref[i][j]+=pref[i][j-1];
                }
                if(i>0 && j>0){
                    pref[i][j]-=pref[i-1][j-1];
                }
            }
        }
        for(int len=1;len<=min(n,m);len++){
            for(int roff=0;roff+len<=n;roff++){
                for(int coff=0;coff+len<=m;coff++){
                    int r1=roff;
                    int c1=coff;
                    int r2=roff+len-1;
                    int c2=coff+len-1;
                    int sum=pref[r2][c2];
                    if(r1>0){
                        sum-=pref[r1-1][c2];
                    }
                    if(c1>0){
                        sum-=pref[r2][c1-1];
                    }
                    if(r1>0 && c1>0){
                        sum+=pref[r1-1][c1-1];
                    }
                    if(sum==len*len){
                        ans=max(ans,sum);
                    }
                }
            }
        }
        return ans;
    }
};
```

```

    }
};

```

Code 3: Maximal Rectangle (Optimal)

Implemented using the Histogram conversion and Stack-based largest area calculation.

```

class Solution {
public:
    int largestarea(int m,vector<int>& his){
        vector<int> prev(m);
        vector<int> next(m);
        stack<int> st;

        for(int i=0;i<m;i++){
            while(!st.empty() && his[st.top()]>=his[i]){
                st.pop();
            }
            if(st.empty()){
                prev[i]=-1;
            }
            else{
                prev[i]=st.top();
            }
            st.push(i);
        }

        while(!st.empty()){
            st.pop();
        }

        for(int i=m-1;i>=0;i--){
            while(!st.empty() && his[st.top()]>=his[i]){
                st.pop();
            }
            if(st.empty()){
                next[i]=m;
            }
            else{
                next[i]=st.top();
            }
            st.push(i);
        }

        int ans=0;
        for(int i=0;i<m;i++){
            int h=his[i];
            int w=next[i]-prev[i]-1;
            ans=max(ans,h*w);
        }
        return ans;
    }

    int maximalRectangle(vector<vector<char>>& matrix) {
        int n=matrix.size();

```

```

int m=matrix[0].size();

vector<vector<int>> his(n,vector<int>(m));
for(int i=0;i<n;i++){
    for(int j=0;j<m;j++){
        if(matrix[i][j]=='0'){
            his[i][j]=0;
        }
        else{
            if(i==0){
                his[i][j]=1;
            }
            else{
                his[i][j]=his[i-1][j]+1;
            }
        }
    }
}

int ans=0;
for(int i=0;i<n;i++){
    ans=max(ans,largestarea(m,his[i]));
}

return ans;
}
};

```